

UNIVERSIDADE DE FORTALEZA
M907 – Sistemas Distribuídos
Prof. Nabor das Chagas Mendonça

RELATÓRIO TÉCNICO – SISTEMAS DISTRIBUÍDOS

Atividade de Laboratório: Implantação e Avaliação Experimental de Aplicações de Microsserviços com Kubernetes

Etapa 1 – Preparação do Ambiente e Implantação das Aplicações

Equipe

Izabella Maria Pinheiro Amorim – Matrícula: 2527553
Letícia Vasconcelos Machado – Matrícula: 2418950
Leonardo de Freitas Rabelo – Matrícula: 2651489
Renan Medeiros Aguiar – Matrícula: 2527541

Fortaleza, junho de 2026

1. Identificação da Equipe

A atividade de laboratório foi desenvolvida pela equipe composta pelos seguintes membros:

Nome	Matrícula	Aplicação Responsável
Izabella Maria Pinheiro Amorim	2527553	Sock Shop
Letícia Vasconcelos Machado	2418950	Sock Shop
Leonardo de Freitas Rabelo	2651489	DeathStarBench
Renan Medeiros Aguiar	2527541	DeathStarBench

Link do Repositório: <https://github.com/izabellamorim/Aplicacoes-Microservicos-de-Codigo-Aberto/commit/f67e6793b7b2f38b4f8c22eb89cb176045a4ab7b>

2. Aplicações Escolhidas

Para esta atividade de laboratório, a equipe selecionou duas aplicações de microserviços previamente analisadas no Trabalho 1: o Sock Shop e o DeathStarBench. A escolha foi motivada pela diversidade de características entre as duas aplicações — uma de perfil mais didático e outra orientada a benchmarking de desempenho — o que permite uma experimentação mais abrangente ao longo das três etapas da atividade.

2.1 Sock Shop

Repositório Principal

<https://github.com/microservices-demo/microservices-demo>

Descrição da Arquitetura

O Sock Shop é uma aplicação de demonstração de e-commerce desenvolvida pela Weaveworks, composta por múltiplos serviços independentes que simulam as operações de uma loja virtual de meias. A aplicação é amplamente utilizada em contextos educacionais e experimentais como referência de implementação de microserviços.

A arquitetura é composta pelos seguintes serviços principais:

- front-end: interface web em Node.js, responsável pela apresentação e integração com os demais serviços via HTTP;
- catalogue: serviço de catálogo de produtos, implementado em Go, com banco de dados MySQL;
- cart: serviço de carrinho de compras, implementado em Java, com Redis para armazenamento em memória;
- orders: serviço de pedidos, implementado em Java, com banco de dados MongoDB;
- user: serviço de gerenciamento de usuários, implementado em Go, com MongoDB;
- payment: serviço de pagamento, implementado em Go (mock de autorização);
- shipping: serviço de frete, implementado em Java (mock de cálculo de envio);

- queue-master: consumidor de fila de mensagens RabbitMQ, responsável por processar pedidos;
- rabbitmq: broker de mensagens utilizado para comunicação assíncrona entre serviços;
- session-db: Redis utilizado para armazenamento de sessões de usuário.

A comunicação entre os serviços ocorre predominantemente via HTTP/REST de forma síncrona, com uso de RabbitMQ para processamento assíncrono de pedidos. Há separação clara entre frontend e backend, e o frontend atua como ponto de entrada e orquestrador das chamadas aos demais serviços.

Principais Serviços

Serviço	Linguagem	Banco de Dados	Função
front-end	Node.js	—	Interface web e API Gateway
catalogue	Go	MySQL	Catálogo de produtos
cart	Java	Redis	Carrinho de compras
orders	Java	MongoDB	Processamento de pedidos
user	Go	MongoDB	Autenticação e cadastro
payment	Go	—	Autorização de pagamento
shipping	Java	—	Cálculo e simulação de frete
queue-master	Java	RabbitMQ	Consumo de fila de pedidos

Fluxo Funcional Escolhido para os Testes

O fluxo funcional definido para os testes simula o comportamento típico de um usuário na loja:

- Acesso à página inicial e listagem de produtos (catalogue);
- Visualização de detalhes de um produto;
- Registro ou login de usuário (user);
- Adição de item ao carrinho (cart);
- Finalização de compra, com acionamento dos serviços de pagamento (payment), pedidos (orders) e frete (shipping).

Esse fluxo percorre os principais serviços da aplicação e é representativo do comportamento real de uso, sendo adequado tanto para testes de desempenho quanto para injeção de falhas em serviços críticos.

Justificativa da Escolha

O Sock Shop foi selecionado por reunir características que o tornam especialmente adequado para as etapas de laboratório:

- Suporte nativo e bem documentado a Kubernetes, com manifestos prontos para implantação;

- Arquitetura de microsserviços representativa, com múltiplos serviços independentes, heterogeneidade tecnológica e diferentes mecanismos de persistência;
- Baixa a média complexidade operacional, facilitando a implantação e execução em ambientes locais (MacBook Pro com Apple M2/ARM64);
- Integração com ferramentas de monitoramento, observabilidade e injeção de falhas, como Prometheus, Kiali e Istio;
- Uso consolidado em pesquisas e disciplinas acadêmicas, com boa disponibilidade de referências e documentação.

Dificuldades de Implantação

A principal dificuldade encontrada no processo de implantação do Sock Shop foi a incompatibilidade de algumas imagens Docker com a arquitetura ARM64 (Apple M2). Algumas imagens do repositório original estavam disponíveis apenas para arquitetura AMD64, exigindo adaptações nos manifestos Kubernetes para utilizar imagens compatíveis ou compiladas para ARM64.

Adicionalmente, foi necessário ajustar configurações de recursos (CPU e memória) nos manifestos para adequação ao ambiente local com recursos limitados em comparação a clusters de produção. Tais adaptações foram documentadas no repositório da equipe.

2.2 DeathStarBench

Repositório Principal

<https://github.com/delimitrou/DeathStarBench>

Descrição da Arquitetura

O DeathStarBench é um conjunto de aplicações de benchmark de microsserviços desenvolvido pela Cornell University, amplamente utilizado em pesquisas acadêmicas para avaliação de desempenho de infraestruturas distribuídas. O conjunto inclui múltiplas aplicações independentes, cada uma representando um domínio diferente.

Para esta atividade, a equipe optou pela aplicação Social Network, que simula uma rede social com as seguintes características arquiteturais:

- Composta por dezenas de serviços independentes, comunicando-se principalmente via Thrift RPC;
- Inclui serviços de composição de posts, linha do tempo, recomendações, notificações e gerenciamento de usuários;
- Utiliza múltiplos bancos de dados, incluindo MongoDB e Redis;
- Suporta execução em Kubernetes com manifestos disponíveis no repositório.

A arquitetura do Social Network é significativamente mais complexa do que o Sock Shop, com maior número de serviços e interdependências, refletindo cenários mais próximos de sistemas distribuídos de larga escala.

Principais Serviços

Serviço	Tecnologia	Função
nginx-thrift	Nginx + Lua	Proxy reverso e roteamento de requisições
compose-post-service	C++ / Thrift	Composição e publicação de posts
post-storage-service	C++ / Thrift	Armazenamento persistente de posts (MongoDB)
user-timeline-service	C++ / Thrift	Gerenciamento da linha do tempo do usuário
home-timeline-service	C++ / Thrift	Linha do tempo agregada (Redis)
user-service	C++ / Thrift	Autenticação e dados de usuário
social-graph-service	C++ / Thrift	Grafo social (seguir/seguidores)
url-shorten-service	C++ / Thrift	Encurtamento de URLs em posts
media-service	C++ / Thrift	Gerenciamento de mídia em posts
text-service	C++ / Thrift	Processamento de texto dos posts
mongodb (múltiplas instâncias)	MongoDB	Persistência de dados por serviço
redis (múltiplas instâncias)	Redis	Cache e armazenamento em memória

Fluxo Funcional Escolhido para os Testes

O fluxo funcional definido para os testes simula a criação e leitura de posts na rede social:

- Registro e autenticação de usuário (user-service);
- Composição de um novo post com texto e URL (compose-post-service → text-service, url-shorten-service);
- Persistência do post (post-storage-service → MongoDB);
- Atualização das linhas do tempo do usuário e dos seguidores (user-timeline-service, home-timeline-service → Redis);
- Leitura da linha do tempo (home-timeline-service).

Esse fluxo cobre os serviços centrais da aplicação Social Network e permite avaliar o comportamento do sistema sob carga de escrita e leitura simultâneas.

Justificativa da Escolha

O DeathStarBench foi selecionado por apresentar características complementares ao Sock Shop:

- Maior complexidade arquitetural, com dezenas de serviços e múltiplas dependências, permitindo experimentos mais sofisticados;
- Uso de comunicação via Thrift RPC, diferindo do padrão REST adotado no Sock Shop e enriquecendo a diversidade dos experimentos;
- Reconhecimento acadêmico consolidado, sendo amplamente utilizado em publicações de sistemas distribuídos e computação em nuvem;
- Disponibilidade de manifestos Kubernetes e scripts de carga próprios, facilitando a configuração dos testes de desempenho.

Dificuldades de Implantação

O DeathStarBench apresentou maior complexidade de implantação em comparação ao Sock Shop. As principais dificuldades encontradas foram:

- Imagens Docker oficiais sem suporte ARM64, exigindo compilação local de algumas imagens para compatibilidade com o ambiente Apple M2;
- Grande número de serviços e dependências, tornando o processo de inicialização mais lento e sujeito a falhas de ordem de inicialização;
- Consumo elevado de recursos computacionais, exigindo ajuste criterioso dos limites de CPU e memória nos manifestos Kubernetes;
- Documentação menos acessível para iniciantes em comparação ao Sock Shop, demandando maior esforço de interpretação dos artefatos do repositório.

Todas as adaptações realizadas foram registradas no repositório da equipe, com indicação das modificações aplicadas aos manifestos originais.

3. Descrição do Ambiente Experimental

Esta seção descreve o ambiente de execução utilizado por cada membro da equipe, incluindo especificações de hardware, sistema operacional e distribuição Kubernetes empregada.

Membro	Sistema Operacional	Processador	RAM	Kubernetes
Izabella	macOS Sequoia 15.x	Apple M2 (ARM64)	16 GB	Docker Desktop
Letícia	macOS Sequoia 15.x	Apple M2 (ARM64)	16 GB	Docker Desktop
Leonardo	Ubuntu 22.04 LTS	Intel Core i7 (x86_64)	16 GB	Minikube
Renan	Ubuntu 22.04 LTS	Intel Core i7 (x86_64)	16 GB	Minikube

3.1 Ambiente de Izabella e Letícia (Sock Shop – macOS / Apple M2)

Sistema Operacional: macOS Sequoia 15.x

Processador: Apple M2 – 8 núcleos (ARM64)

Memória RAM: 16 GB LPDDR5

Distribuição Kubernetes: Kubernetes integrado ao Docker Desktop

Versão do Kubernetes: v1.29.x

Docker Desktop: versão 4.x

kubecti: v1.29.x

Aplicação testada: Sock Shop

O ambiente de Izabella e Letícia utiliza o Kubernetes integrado ao Docker Desktop, que provê um cluster de nó único diretamente no macOS sem necessidade de máquina virtual separada.

Essa configuração é conveniente para uso local, mas impõe limitações de recursos em relação a ambientes de produção. A principal adaptação necessária para o ambiente ARM64 foi a substituição de imagens Docker não compatíveis com a arquitetura Apple Silicon.

3.2 Ambiente de Leonardo e Renan (DeathStarBench – Ubuntu / x86_64)

Sistema Operacional: Ubuntu 22.04 LTS

Processador: Intel Core i7 (x86_64, 8 núcleos)

Memória RAM: 16 GB DDR4

Distribuição Kubernetes: Minikube

Versão do Kubernetes: v1.29.x

Versão do Minikube: v1.32.x

Driver Minikube: Docker

kubectrl: v1.29.x

Aplicação testada: DeathStarBench (Social Network)

O ambiente de Leonardo e Renan utiliza Minikube com driver Docker em Ubuntu 22.04 LTS. A arquitetura x86_64 facilitou a utilização das imagens Docker oficiais do DeathStarBench sem necessidade de recompilação. O uso de Minikube com perfil de recursos ajustado (CPUs e memória) foi necessário para viabilizar a execução simultânea de todos os serviços do Social Network em ambiente local.

4. Relato do Processo de Implantação

4.1 Sock Shop – Izabella e Letícia

4.1.1 Obtenção dos Artefatos

O processo de implantação do Sock Shop foi iniciado com a clonagem do repositório oficial:

```
git clone https://github.com/microservices-demo/microservices-demo.git
```

Os manifestos Kubernetes estão localizados no diretório `deploy/kubernetes/` do repositório, contendo o arquivo `complete-demo.yaml` com todos os recursos necessários para implantação em um único comando.

4.1.2 Adaptações Realizadas

As seguintes adaptações foram necessárias para execução no ambiente Apple M2 (ARM64) com Docker Desktop:

- Substituição das imagens Docker sem suporte ARM64 por equivalentes multi-arch ou imagens compiladas localmente;
- Redução dos requests e limits de CPU e memória em alguns Deployments para adequação ao ambiente local com recursos compartilhados;
- Ajuste de `imagePullPolicy` para `Always` em serviços com imagens personalizadas;
- Habilitação do namespace dedicado `sock-shop` para isolamento dos recursos.

4.1.3 Processo de Implantação

Com as adaptações realizadas, a implantação foi executada com os seguintes comandos:

```
kubectl create namespace sock-shop
kubectl apply -f deploy/kubernetes/complete-demo.yaml
kubectl get pods -n sock-shop --watch
```

Após aguardar que todos os pods atingissem o estado Running, o acesso à aplicação foi verificado via port-forward na porta 30001, com acesso pelo navegador em <http://localhost:30001>.

4.1.4 Verificação de Funcionamento

A aplicação foi verificada como funcional por meio dos seguintes cenários básicos:

- Acesso à página inicial e listagem do catálogo de produtos;
- Cadastro de novo usuário e autenticação;
- Adição de produto ao carrinho de compras;
- Finalização de um pedido simulado.

Todos os fluxos foram executados com sucesso, confirmando o correto funcionamento da aplicação no ambiente local. Capturas de tela e saídas de kubectl foram registradas e incluídas no repositório da equipe.

4.2 DeathStarBench – Leonardo e Renan

4.2.1 Obtenção dos Artefatos

O repositório do DeathStarBench foi clonado a partir do GitHub oficial:

```
git clone https://github.com/delimitrou/DeathStarBench.git
```

Os manifestos Kubernetes para a aplicação Social Network estão localizados no diretório `socialNetwork/helm-chart/`, com suporte à implantação via Helm. Também estão disponíveis manifestos YAML nativos no diretório `socialNetwork/kubernetes/`.

4.2.2 Preparação do Cluster Minikube

Antes da implantação, foi necessário inicializar o Minikube com recursos suficientes para suportar todos os serviços do Social Network:

```
minikube start --cpus=6 --memory=12288 --driver=docker
```

4.2.3 Adaptações Realizadas

As seguintes adaptações foram realizadas nos artefatos originais:

- Redução dos resource requests de alguns serviços para viabilizar execução no ambiente local;
- Ajuste de réplicas para 1 em todos os Deployments durante a implantação inicial;
- Configuração de InitContainers para garantir a ordem correta de inicialização dos serviços dependentes de MongoDB e Redis;
- Adição de readinessProbes em serviços que não as possuíam, para maior controle da ordem de disponibilização.

4.2.4 Processo de Implantação

A implantação foi realizada utilizando os manifestos Kubernetes nativos, com namespace dedicado:

```
kubectl create namespace social-network
kubectl apply -f socialNetwork/kubernetes/ -n social-network
kubectl get pods -n social-network --watch
```

A inicialização completa de todos os serviços demandou aproximadamente 5 a 7 minutos, devido ao grande número de containers e à necessidade de download das imagens. O acesso à aplicação foi verificado via port-forward na porta do nginx-thrift.

4.2.5 Verificação de Funcionamento

A aplicação Social Network foi verificada como funcional por meio dos seguintes cenários básicos:

- Registro de usuário via API REST (nginx-thrift);
- Criação de post com texto e URL;
- Leitura da linha do tempo do usuário;
- Operações de seguir e ser seguido (social graph).

Todas as operações foram executadas com sucesso utilizando os scripts de teste fornecidos pelo DeathStarBench, confirmando o correto funcionamento da aplicação. Os resultados foram registrados e evidenciados no repositório da equipe.

5. Adaptações nos Artefatos Originais

Esta seção consolida as principais adaptações realizadas pela equipe nos manifestos e artefatos originais das duas aplicações.

5.1 Adaptações no Sock Shop

Artefato	Modificação	Justificativa
complete-demo.yaml	Substituição de imagens AMD64 por ARM64	Compatibilidade com Apple M2
Deployments (recursos)	Redução de CPU requests/limits	Adequação ao ambiente local com recursos limitados
Deployments (recursos)	Redução de memory requests/limits	Prevenção de OOMKill em ambiente com 16GB compartilhado
Namespace	Criação de namespace sock-shop	Isolamento dos recursos da aplicação

5.2 Adaptações no DeathStarBench

Artefato	Modificação	Justificativa
Deployments (réplicas)	Redução para 1 réplica por serviço	Redução do consumo de recursos no cluster local

Artefato	Modificação	Justificativa
Deployments (recursos)	Ajuste de CPU e memory requests/limits	Viabilização da execução no Minikube com 12GB alocados
InitContainers	Adição de initContainers de espera	Garantir ordem de inicialização correta dos serviços dependentes
ReadinessProbes	Adição de readinessProbes	Controle de disponibilidade e estabilidade na inicialização
Namespace	Criação de namespace social-network	Isolamento dos recursos da aplicação

6. Evidências de Funcionamento

As evidências de funcionamento das duas aplicações foram coletadas durante e após o processo de implantação. Esta seção descreve as evidências registradas, que foram incluídas no repositório da equipe.

6.1 Sock Shop

As seguintes evidências foram coletadas e incluídas no repositório:

- Captura de tela da listagem de pods em estado Running (`kubectl get pods -n sock-shop`);
- Captura de tela da interface web da loja acessível em `http://localhost:30001`;
- Captura de tela de cadastro e autenticação de usuário;
- Captura de tela de adição de produto ao carrinho e finalização de compra;
- Saída do comando `kubectl get services -n sock-shop` com todos os serviços listados;
- Logs dos pods principais confirmando inicialização sem erros críticos.

6.2 DeathStarBench

As seguintes evidências foram coletadas e incluídas no repositório:

- Captura de tela da listagem de pods em estado Running (`kubectl get pods -n social-network`);
- Saída da execução de operações via API: registro de usuário, criação de post e leitura de timeline;
- Saída do comando `kubectl get services -n social-network`;
- Logs do nginx-thrift e dos serviços de composição e armazenamento de posts;
- Execução do script de geração de carga nativo do DeathStarBench confirmando conectividade com os serviços.

7. Conteúdo do Repositório

O repositório da equipe foi atualizado com os artefatos correspondentes à Etapa 1. A estrutura adotada segue a recomendação do enunciado, com os seguintes conteúdos disponíveis:

```
/ ├── README.md ├── report/ |   ├── relatorio_etapa1.pdf ├── apps/ |   ├── sock-shop/ |   |   ├── manifests/ |   |   |   ├── complete-demo-arm64.yaml |   |   |   ├── social-network-kubernetes/ |   |   |   ├── screenshots/
```

O README do repositório contém instruções detalhadas para: preparação do ambiente em macOS (Apple M2) e Ubuntu (x86_64), implantação das duas aplicações e verificação de funcionamento básico.

8. Próximas Etapas

Com a conclusão bem-sucedida da Etapa 1, a equipe está preparada para avançar para a Etapa 2, que compreende a realização dos testes de desempenho nas duas aplicações.

As atividades planejadas para a Etapa 2 incluem:

- Definição dos scripts de carga com Locust para ambas as aplicações;
- Calibração dos três níveis de carga (baixa, média e alta) para cada aplicação;
- Execução dos testes de desempenho por cada membro da equipe em seus respectivos ambientes;
- Coleta e registro das métricas obrigatórias: tempo médio de resposta, P95, taxa de falha e número de requisições;
- Produção de gráficos comparativos e análise dos resultados obtidos.

Para os testes de resiliência (Etapa 3), a equipe já identificou os serviços críticos candidatos à injeção de falhas: o serviço front-end ou payment no Sock Shop, e o compose-post-service no DeathStarBench. A justificativa e o detalhamento dessas escolhas serão apresentados no relatório da Etapa 3.

Referências

DELIMITROU, Christina; KOZYRAKIS, Christos. DeathStarBench: A Benchmark Suite for Microservices. GitHub. Disponível em: <https://github.com/delimitrou/DeathStarBench>. Acesso em: jun. 2026.

WEAVEWORKS. Sock Shop Microservices Demo Application. GitHub. Disponível em: <https://github.com/microservices-demo/microservices-demo>. Acesso em: jun. 2026.

WEAVEWORKS. Sock Shop Documentation. Disponível em: <https://microservices-demo.github.io>. Acesso em: jun. 2026.

KUBERNETES. Kubernetes Documentation. Disponível em: <https://kubernetes.io/docs>. Acesso em: jun. 2026.

NEWMAN, Sam. Building Microservices: Designing Fine-Grained Systems. 2. ed. Sebastopol: O'Reilly Media, 2021.

DOCKER. Docker Desktop for Apple Silicon. Disponível em: <https://docs.docker.com/desktop/mac/apple-silicon/>. Acesso em: jun. 2026.